DJ Set-Planning App Production Document

Target Audience: Anti-Gravity (Production Team)

1. Core Concept

A set-progression web algorithm that translates the artistic 'feel' of a DJ set into programmable logic. The app accepts a user's playlist file and re-orders the tracks based on tension, BPM, and harmonic mixing rules.

2. DJ Theory & Logic

- **BPM (Tempo Progression):** Groups tracks by BPM to create gradual tempo ramps or distinct tempo blocks.
- **Harmonic Mixing (Camelot Wheel):** Prevents musical clashes. Maintains smooth flow by keeping keys identical or adjacent (e.g., 8A to 8A/9A). Builds tension using an 'Energy Boost' (jumping two steps up, e.g., 8A to 10A).
- **Energy Levels & The 'Curve':** Manages the macro-tension of the set based on user-selected shapes:
    - *The Ramp:* Steady build-up to a high-energy peak.
    - *The Wave (Rollercoaster):* Alternates between peaks and low-energy breathers.
    - *The Flatline:* Consistent background energy (for opening/lounge sets).

3. Technical Architecture

- **Input Parsing:** Accepts `.m3u` or `.xml` (Rekordbox standard), or `.csv`/`.txt` (Spotify). Requires a script to extract Track Title and Artist.
- **Data Enrichment (Spotify Web API):** Uses the 'Audio Features' endpoint to fetch exact BPM, Musical Key, Energy score (0.0 - 1.0), and Danceability for each track.

- **The Sorting Engine:**
    a. Reads user's chosen Energy Curve.
    b. Selects an appropriate starting track.
    c. Loops through remaining tracks to find matches with compatible Camelot Keys, similar BPM (+/- 2), and the correct energy trajectory.

4. Phase-by-Phase Implementation Plan

Phase	Milestone	Core Focus & Deliverables
Phase 1	MVP	Text-based input for 10 songs. Connects to Spotify API to fetch and display BPM/Key in a table.
Phase 2	Basic Ordering	Implements BPM sorting and flags harmonic (key) clashes.
Phase 3	Tension Algorithm	Adds Energy Curve logic to automate set arrangement based on chosen shapes.
Phase 4	File Uploads	Implements `.xml` and `.m3u` file parsing for direct DJ software integration.

5. Playlist Parsing Implementation

- **M3U Parsing:**
    - *Python:* Read the file line-by-line using standard file I/O. Ignore lines starting with `#` (unless extracting `#EXTINF` for metadata) to isolate the track names or file paths.
    - *JavaScript:* Read the text string, split by the newline character (`\n`), and filter out comment lines.

- **XML (Rekordbox) Parsing:**
    - *Python:* Use the built-in `xml.etree.ElementTree` library to target the `<COLLECTION>` node and iterate through individual `<TRACK>` elements, extracting attributes like `Name` and `Artist`.
    - *JavaScript:* In a browser, use the native `DOMParser` API. In a Node.js backend environment, utilize packages like `xml2js` or `fast-xml-parser` to convert the XML into a manipulatable JSON object.

6. Spotify API Integration (Data Enrichment)

- **Authentication:** The app will use the Client Credentials authorization flow to obtain an access token, as track analysis does not require user-specific data.
- **Endpoint:** A `GET` request to the `https://api.spotify.com/v1/audio-features/{id}` endpoint (or the bulk endpoint for multiple track IDs).
- **Data Mapping:**
    - **BPM:** Mapped from the `tempo` response field (estimated overall tempo in beats per minute).
    - **Musical Key:** Mapped from the `key` response field (returns an integer mapping to standard Pitch Class notation, which the app will translate to the Camelot Wheel).
    - **Energy Level:** Mapped from the `energy` response field (a perceptual measure of intensity from 0.0 to 1.0).

7. Local Execution Strategy

- **Application Format:** Command Line Interface (CLI) application for direct execution without a web server.
- **Primary Language:** Python, due to native libraries for data handling and straightforward API integrations.
- **Environment Setup (macOS):**
    - Run directly via the macOS Terminal.

    - Use a Python virtual environment (`python3 -m venv venv`) to isolate dependencies.
    - Require the `requests` library (`pip install requests`) to handle Spotify API calls.
- **Execution Flow:** The script will accept a playlist file path as a command-line argument (e.g., `python3 dj_sorter.py ~/Downloads/playlist.xml`), parse the file, query the Spotify API, calculate the tension curve, and output the sorted tracklist directly in the terminal or write it to a new file.